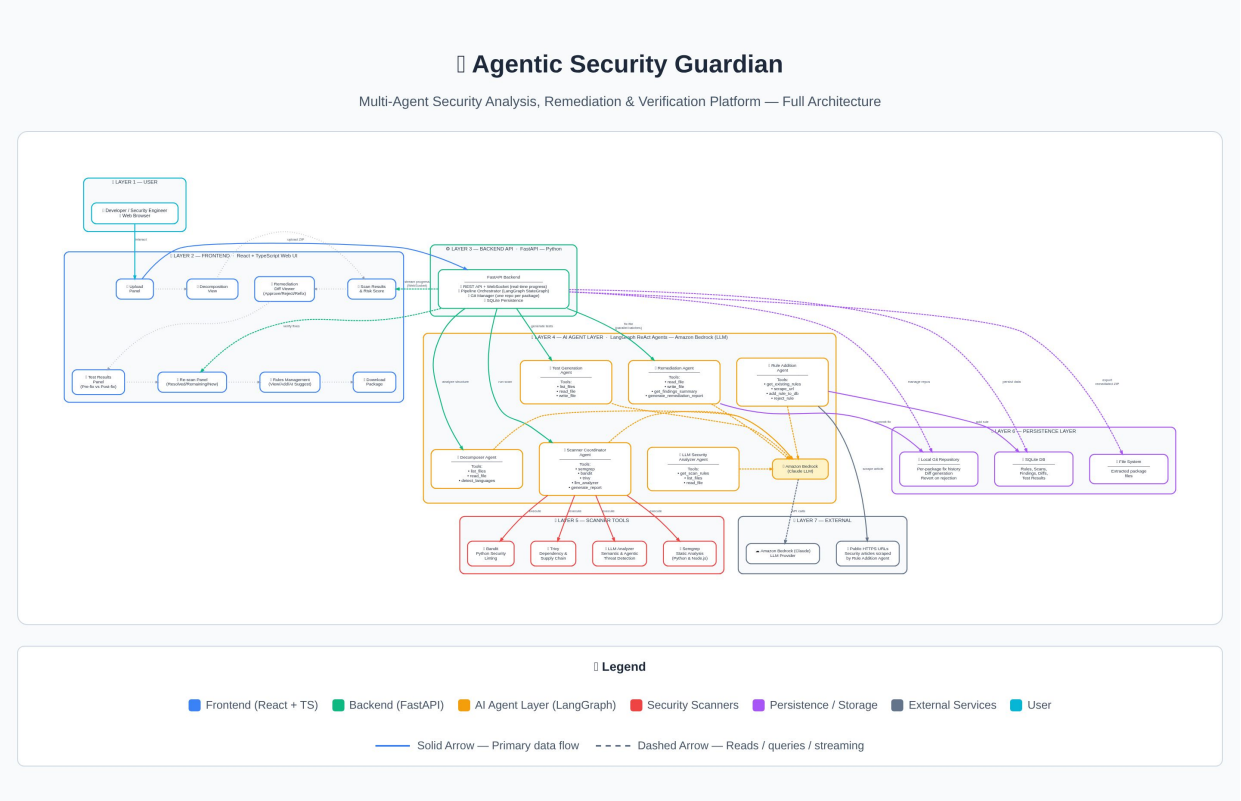
AI security frameworks

Scope

Agentic Security Guardian is an end-to-end automated security platform for analyzing and remediating vulnerabilities in application code packages. It covers multi-language static analysis using Semgrep, Bandit, and Trivy, LLM-based semantic analysis for agentic and AI-specific threats, intelligent package decomposition, autonomous fix generation with human-in-the-loop approval, functional test generation pre and post fix, re-scanning to verify remediation effectiveness, and export of the fully remediated package. The platform supports an extensible rules knowledge base of 29 built-in security rules that can be grown via manual input or AI-assisted extraction from public security articles.

Architecture

Architecture Diagram:



- **Frontend — React + TypeScript**
    - Single-page application for all user interactions
    - Communicates with backend via REST API and WebSocket for real-time scan progress
    - Key views: Upload, Decomposition, Scan Results, Remediation Diffs, Rules Management, Test Results, Re-scan Panel, Download
- **Backend — FastAPI (Python)**
    - Central orchestrator coordinating the full pipeline via a LangGraph StateGraph
    - Exposes REST endpoints for upload, decompose, scan, remediation, rules, and export
    - Manages a local git repository per package to track all agent-made changes and enable diff generation and revert on rejection
    - Persists all state in SQLite — rules, scan history, scan results, diffs, test results
- **AI Agent Layer — 6 LangGraph ReAct Agents**
    - Decomposer Agent — reads package structure, detects languages, determines intent and recommends which scanners to run
    - Scanner Coordinator Agent — orchestrates Semgrep, Bandit, Trivy and LLM analyzer tools, collects all findings and generates the scan report
    - LLM Security Analyzer Agent — reads source files against active security rules, performs semantic and agentic threat detection
    - Remediation Agent — generates remediation report from findings, fixes vulnerable files, commits changes to local git repo
    - Test Generation Agent — reads source files and writes pytest functional test cases to the package
    - Rule Addition Agent — scrapes public HTTPS URLs, validates content for malicious intent, checks for duplicates, extracts and adds new rules to the DB
    - All agents use Amazon Bedrock as the LLM provider
- **Security Scanner — 4 Tools**
    - Semgrep — static analysis for Python and Node.js
    - Bandit — Python-specific security linting
    - Trivy — dependency and supply chain vulnerability scanning
    - LLM Analyzer — semantic analysis for agentic, AI-specific, and logic-level threats using active rules from the knowledge base
- **Storage**

    - SQLite DB — rules knowledge base, scan history, findings, diffs, test results
    - Local filesystem — extracted package files per upload
    - Local git repo — one repo per package, every agent fix committed as a trackable change
- **External Dependencies**
    - Amazon Bedrock — LLM provider for all six agents
    - PyPI / Semgrep / Trivy rule sets — scanner dependencies
    - Public HTTPS URLs — scraped by Rule Addition Agent for rule extraction

UC's

- **UC1 — Upload and decompose a code package**
  A user uploads a ZIP of their application. The Decomposer Agent analyzes the file structure, detects languages, understands the application intent, and recommends which scanners are applicable.
- **UC2 — Scan a package for vulnerabilities**
  The system runs all recommended scanners — Semgrep, Bandit, Trivy, and LLM Analyzer — in parallel, collects all findings, and presents them grouped by severity with an overall risk score.
- **UC3 — Generate functional test cases**
  Before the scan begins, the Test Generation Agent reads the source code and writes pytest unit tests that verify the application's functional behaviour. These serve as a baseline.
- **UC4 — Automatically remediate vulnerabilities**
  After reviewing findings, the user triggers remediation. The Remediation Agent fixes each vulnerable file, commits changes to a local git repo, and presents a diff per file for human review.
- **UC5 — Approve or reject a fix with feedback**
  The user reviews each generated fix diff. They can approve it or reject it with written feedback — triggering the agent to refix based on that feedback.
- **UC6 — Re-run all fixes from scratch**
  The user can reset and re-trigger remediation for all files at once.
- **UC7 — Verify fixes via re-scan**
  After approving fixes, the user triggers a re-scan. The system re-runs the security scanners on the fixed code and reports how many vulnerabilities were resolved, how many remain, and whether any new issues were introduced.
- **UC8 — Pre and post fix functional testing**
  The generated test cases are run before fixes as a baseline and again after all fixes are complete — ensuring security remediation did not break existing functional behaviour.
- **UC9 — Download the remediated package**
  The user downloads a ZIP of the fixed codebase including all approved changes, companion files like .env, and the generated test cases.
- **UC10 — Add a custom security rule manually**
  A security engineer adds a new rule by entering a category, name, group, description, and severity. The rule is immediately active for all future scans.
- **UC11 — AI-assisted rule extraction from a security article**
  A security engineer provides a URL to a public security article or advisory. The Rule Addition Agent scrapes the page, validates the content for malicious intent and prompt injection attempts, checks for duplicates against all existing rules, and automatically adds only genuinely new and specific rules to the knowledge base.

Demo Link

Agentic Security Guardian - Demo Recording-20260316_123752-Meeting Recording.mp4

Presentation link

1. Agentic_Security_Guardian.pptx
2. Agentic Security Guardian- Flow.pptx